

# Victor's Task Program

Everything you, not Claude, have to do, written like code: explicit inputs, numbered steps, IF/ELSE branches, expected output and what to do on error. Work through it slowly, one procedure at a time.

|    |                                              |                   |
|----|----------------------------------------------|-------------------|
| 00 | How to run this program (read first)         | DO NOW            |
| 01 | answer_phase2_questions()                    | ✓ DONE 3 Oct      |
| 02 | confirm_school_rules()                       | ✓ DONE 9 Oct      |
| 03 | test_school_as_learner()                     | DO NOW · weekends |
| 04 | bring_back_ui_design()                       | WHEN READY        |
| 05 | deploy_on_vercel()                           | ✓ DONE 9 Oct      |
| 06 | connect_subdomain()                          | ✓ DONE 9 Oct      |
| 07 | create_supabase_project()                    | ON HOLD           |
| 08 | set_up_resend()                              | ON HOLD           |
| 09 | deploy_email_nudges()                        | ON HOLD           |
| 10 | Appendix: words, tools and message templates |                   |

**Why some procedures are ON HOLD.** You put hosting and the backend on hold. They're written out in full so they're ready when you decide. **Doing 07–09 today won't change anything you can see,** because the app doesn't log people in yet. Claude wires accounts into the app after your UI design comes back.

## 00 · How to run this program

Read this page once. It works like the rules of a programming language: every procedure after it follows them.

### The rules

- **01**    **One procedure at a time.** Finish it, or stop it cleanly, before starting another. Never run two at once.
- **02**    **Run the steps in order.** Don't skip a step because it "looks obvious". If a step says *copy*, copy; don't retype.
- **03**    **Tick each** □ with a pen as you finish it. If you stop halfway, the last tick tells you where to resume.
- **04**    **Check the EXPECTED OUTPUT** at the end of each procedure. If what you see matches, the procedure is done.
- **05**    **ON ERROR, stop.** Don't improvise. Take a screenshot, then send Claude the error template from the Appendix (section 10).
- **06**    **Secrets never go into chat, email, WhatsApp or the repo.** A secret is any value this document labels **SECRET**. Keep them in a password manager (Apple Passwords, 1Password or Bitwarden).

### How to read a procedure

|                        |                                                                             |
|------------------------|-----------------------------------------------------------------------------|
| <b>PROCEDURE</b>       | The name of the task, like a function name.                                 |
| <b>WHY</b>             | What it achieves, in one sentence.                                          |
| <b>INPUTS</b>          | What you need before you start. If you don't have every input, don't start. |
| <b>TIME</b>            | Roughly how long it takes.                                                  |
| <b>STEPS</b>           | Numbered actions, done in order.                                            |
| <b>IF / ELSE</b>       | A fork: do the branch that matches what you see.                            |
| <b>EXPECTED OUTPUT</b> | What "done" looks like.                                                     |
| <b>ON ERROR</b>        | What to do when it doesn't work.                                            |

### Notation

- **Bold text in steps** = a button, menu or label to click on screen, spelt exactly as it appears.
- Grey monospace = text to type or paste exactly, character for character.
- <LIKE\_THIS> = a placeholder. Replace the whole thing, including the < >, with your own value.
- → = "then click". **Settings** → **Domains** means click Settings, then click Domains.
- "Terminal" = the Mac app called Terminal (press ⌘ + **Space**, type `Terminal`, press Enter).

## 01 · answer\_phase2\_questions() DO NOW

PROCEDURE answer\_phase2\_questions() → 3 answers

|        |                                                                                                                                            |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------|
| WHY    | Fullstack Weeks 11–14 can't be written until you decide these. Weeks 1–10 are written and live.                                            |
| INPUTS | 15 quiet minutes. Optional: the draft plan at <code>src/content/courses/fullstack-ml/phase-2-plan.md</code> (on GitHub: repo → that path). |
| TIME   | 15 minutes                                                                                                                                 |

- 01 **Question 1: order.** The draft is: Week 11 modern CSS (Grid, multi-page sites) → Weeks 12–13 React (components, then state and fetching) → Week 14 milestone in plain HTML/CSS/JS. Decide:  
**IF** you're happy with that order → answer `Order: keep`  
**ELSE IF** you want React earlier and the milestone built in React → answer `Order: React first`
- 02 **Question 2: milestone business.** Week 14's milestone is a multi-page website (Home · Menu · About · Contact) for a pretend Nigerian business. The suggestion is "**Mama Put Kitchen**", a Lagos buka: menu with ~~N~~ prices, opening hours, WhatsApp ordering. Alternatives: an Abuja tailor, an Enugu phone-repair shop.  
**IF** you like it → `Business: Mama Put Kitchen`  
**ELSE** → `Business: <YOUR_IDEA>` plus one sentence about what it sells.
- 03 **Question 3: Saturdays.** As drafted, the Saturdays of Weeks 11–14 all build towards the milestone site. The alternative is one separate project each Saturday.  
**IF** yes → `Saturdays: build the milestone` **ELSE** → `Saturdays: separate mini-projects`
- 04 Open Claude Code (Mac or `claude.ai/code`) in this project.
- 05 Paste this message, with your three answers filled in, and send it:

```
Phase 2 decisions:
Order: <keep | React first>
Business: <Mama Put Kitchen | your idea + one sentence>
Saturdays: <build the milestone | separate mini-projects>
Write Fullstack Weeks 11–14 when it's a weekday.
```

**EXPECTED OUTPUT:** Claude replies confirming the plan. On the next weekday it writes Weeks 11–14 and pushes them to the test site.

**ON ERROR:** If you can't decide, send `Phase 2: use your defaults`. The defaults are: keep the order, Mama Put Kitchen, Saturdays build the milestone.

## 02 · confirm\_school\_rules()

DO NOW

PROCEDURE confirm\_school\_rules() → 5 decisions

|        |                                                                                                                                                                               |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WHY    | The app runs on default rules that you haven't approved yet. Each rule lives in one place in the code, so changing it later is cheap, but approving them now stops guesswork. |
| INPUTS | The table below.                                                                                                                                                              |
| TIME   | 10 minutes                                                                                                                                                                    |

For each row, write **KEEP** or **CHANGE: <how>** in the last column.

| Rule                      | Current default                                                                                                                                                                                    |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| R1 Pacing                 | "Today" = your first unfinished day. Missing an evening never puts you behind. <input type="checkbox"/> KEEP <input type="checkbox"/> CHANGE: _____                                                |
| R2 Week quiz pass mark    | The end-of-week quiz is self-marked; you pass when every question is answered and marked "got it". (Daily gates stay at 70%.) <input type="checkbox"/> KEEP <input type="checkbox"/> CHANGE: _____ |
| R3 Streak                 | Counts Mon–Sat study days. Sunday is rest and never breaks or adds to it. <input type="checkbox"/> KEEP <input type="checkbox"/> CHANGE: _____                                                     |
| R4 "Need a nudge?" button | No behaviour yet. Ideas: send a reminder email in 1 hour · show the 10-minute version of today · open a motivational note. <input type="checkbox"/> pick: _____                                    |
| R5 Who writes new weeks   | Claude writes them in Claude Code, a slice at a time, verified before pushing. <input type="checkbox"/> KEEP <input type="checkbox"/> CHANGE: _____                                                |

☐ 01 Fill in all five rows on paper.

☐ 02 Send Claude:

```
School rules:
R1 pacing: <KEEP | CHANGE: ...>
R2 week quiz: <KEEP | CHANGE: ...>
R3 streak: <KEEP | CHANGE: ...>
R4 nudge button: <your pick>
R5 writing weeks: <KEEP | CHANGE: ...>
```

**EXPECTED OUTPUT:** Claude moves these from "Open decisions" to "Decided" in CLAUDE.md, and changes code only for rows you marked CHANGE.

## PROCEDURE test\_school\_as\_learner(course) → bug reports

|        |                                                                                           |
|--------|-------------------------------------------------------------------------------------------|
| WHY    | You're the first real learner. Things you trip over will trip up every learner after you. |
| INPUTS | Your phone <b>and</b> a laptop. The school: <code>school.scribsandpixels.com</code>       |
| TIME   | 20–40 minutes per session. Repeat any weekend.                                            |

**Know this first:** progress is saved **in this browser only**. A different phone or browser, a private tab, or clearing site data starts you from zero. That's expected until accounts exist.

**Checkpoints are strict:** once you press **Start checkpoint**, leaving, reloading or closing the page counts as a failed attempt (60-minute cooldown). Start only when you have 10 calm minutes.

- **01** Open the test site on your phone.
- **02** Tap one course. Choose one per session: Fullstack, French or Excel.
- **03** Open the week you're actually on, then today's day.
- **04** Do the day **as a learner**: read, open the links, tick blocks, do the quiz, use any sandbox, write the log.
- **05** While doing it, note anything in these 5 bug types:
  - **BROKEN**: an error, a blank screen, a button that does nothing.
  - **WRONG**: a wrong quiz answer, a wrong number, a typo, a dead link.
  - **CONFUSING**: you didn't know what to do next.
  - **UGLY**: overlapping text, too small on the phone, bad colours.
  - **SLOW**: you waited more than 3 seconds.
- **06** For each bug: take a screenshot, then copy the address bar (the bit after `#/` says exactly which page).
- **07** Optional, to test checkpoints quickly: tap the round  button (bottom right of the screen) → **Force checkpoint on this day**. To skip a 60-minute cooldown: **End cooldown now**.
- **08** **IF** you want to start a course again →  → **Reset my progress** → **OK**. **ELSE** leave it.
- **09** Send all bugs from the session in one message, using the bug template in section 10.

**EXPECTED OUTPUT:** One message to Claude listing your bugs. Claude fixes them and replies with what changed.

**ON ERROR:** If the whole site is blank, reload once. If it's still blank, try another browser. If it's blank everywhere, send `Site is blank on <device/browser>` with a screenshot.

## 04 · bring\_back\_ui\_design()

WHEN READY

PROCEDURE `bring_back_ui_design()` → design handed to Claude Code

|        |                                                                                                                                                 |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| WHY    | The test site has plain screens. The real look is being designed on claude.ai. Claude Code can only build it once the design is in the project. |
| INPUTS | Your finished design conversation or artifact on claude.ai.                                                                                     |
| TIME   | 15 minutes                                                                                                                                      |

- 01 On claude.ai, open the design conversation.
- 02 Ask it: Give me the final design as one HTML file plus a list of every screen and what each button does.
- 03 **IF** it produced an artifact → click it → **Share / Publish** (or the link icon) → copy the link.  
**ELSE** → copy the whole HTML it gave you into a file named `design.html`.
- 04 In Claude Code, send:

Here is the final UI design: <LINK or attach design.html>.  
Screens and buttons: <paste the list>.  
Build it, replacing the test screens. Keep all content, quizzes and sandboxes working.

**EXPECTED OUTPUT:** Claude confirms it can read the design and plans the build. Building happens on weekdays.

**ON ERROR:** If Claude says it can't open the link, attach the HTML file instead (step 3, ELSE branch).

**PROCEDURE** `deploy_on_vercel()` → a second live copy of the site

|               |                                                                                            |
|---------------|--------------------------------------------------------------------------------------------|
| <b>WHY</b>    | Vercel is faster and handles custom domains easily. GitHub Pages keeps working either way. |
| <b>INPUTS</b> | Your GitHub login (account <b>VictorAnimasahun</b> ).                                      |
| <b>TIME</b>   | 10 minutes                                                                                 |

- **01** Go to `vercel.com` → **Sign Up** → **Continue with GitHub**. Choose the free **Hobby** plan.
- **02** On the dashboard: **Add New...** → **Project**.
- **03** Find `scribsandpixels-school` in the list → **Import**.  
**IF** it isn't listed → **Adjust GitHub App Permissions** → allow the repo → come back.
- **04** Check the settings. Change only what's different:
  - Framework Preset: `Vite`
  - Build Command: `npm run build`
  - Output Directory: `dist`
  - Environment Variables: **none**. Leave the section empty.
- **05** Click **Deploy**. Wait for the confetti (about 1–2 minutes).
- **06** Click the preview image. A new tab opens on a `...vercel.app` address.
- **07** Check that the home page shows the three courses, then open any day.
- **08** Send Claude: Vercel is live at `<URL>`

**EXPECTED OUTPUT:** The same school at a `.vercel.app` address. Every push to `main` redeploys it automatically.

**ON ERROR:** If the build fails, open the build log, scroll to the first red line, screenshot it and send it to Claude. Don't change settings by guessing.

**PROCEDURE** connect\_subdomain(domain) → school.<yourdomain>

|               |                                                                                                                                                                                                |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>WHY</b>    | A clean address like school.scribsandpixels.com instead of the long GitHub one.                                                                                                                |
| <b>INPUTS</b> | (a) 05 done. (b) A login to the company that holds your domain (the "registrar" or DNS host, e.g. Namecheap, GoDaddy, Cloudflare, Whogohost). (c) The exact domain and the subdomain you want. |
| <b>TIME</b>   | 15 minutes, then up to 24 hours for the internet to catch up                                                                                                                                   |

- **01** Decide the address, e.g. school.<yourdomain>. Done: school.scribsandpixels.com
- **02** In Vercel: open the project → **Settings** → **Domains** → type the address → **Add**.
- **03** Vercel shows a record to create. It's usually: Type **CNAME**, Name school, Value cname.vercel-dns.com. Copy what it shows if that's different.
- **04** Log in to your registrar → find **DNS / DNS Records / Advanced DNS** for the domain.
- **05** **Add record** → fill in exactly the Type, Name and Value from step 3. Leave TTL on Automatic → **Save**.  
**IF** your registrar is Cloudflare → set the orange cloud (proxy) to **grey / DNS only**.
- **06** Back in Vercel → **Domains**. Wait until the status says **Valid Configuration** (a refresh every few minutes is fine).
- **07** Open the new address in your phone browser.
- **08** Send Claude: Subdomain live: <address>. Make it the main host and update APP\_URL references.

**EXPECTED OUTPUT:** The school opens at your subdomain with a padlock (https).**ON ERROR:** "Invalid configuration" after 24 hours → screenshot the registrar's DNS page and Vercel's Domains page, and send both. **Don't delete other DNS records:** they run your email and main website.

## 07 · create\_supabase\_project() ON HOLD

PROCEDURE create\_supabase\_project() → database with tables + 2 keys

|        |                                                                                                                                                   |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| WHY    | Supabase will store learners' accounts and progress, so progress follows you across devices. Only useful once Claude wires accounts into the app. |
| INPUTS | A GitHub login, a password manager, and the Mac with this project.                                                                                |
| TIME   | 25 minutes                                                                                                                                        |

- 01 Go to `supabase.com` → **Start your project** → **Continue with GitHub**.
- 02 **New project**. Name: `scribsandpixels-school`. Region: **West EU (London)** or **Central EU (Frankfurt)**, the closest to Lagos. Plan: **Free**.
- 03 Database Password: click **Generate a password** → save it in your password manager as **SECRET** "Supabase DB password" → **Create new project**. Wait about 2 minutes.
- 04 Open the Terminal and install the Supabase tool (one time only):

```
brew install supabase/tap/supabase
```

**IF** it says `brew: command not found` → stop and send that error to Claude (Homebrew needs installing first).
- 05 Log in from the Terminal (a browser window opens; approve it):

```
supabase login
```
- 06 In Supabase: **Project Settings** (gear icon) → **General** → copy **Project ID** (the "project ref", 20 letters).
- 07 In the Terminal, go to the project and link it (paste your ref; type the DB password when asked):

```
cd ~/Desktop/scribsandpixels-school
supabase link --project-ref <PROJECT_REF>
```
- 08 Create all the tables (this runs the migration file Claude wrote):

```
supabase db push
```

When asked `Do you want to push these migrations?` type `Y` and press Enter.
- 09 In Supabase: **Table Editor**. You should see tables including `profiles`, `enrollments` and `quiz_attempts`.
- 10 In Supabase: **Project Settings** → **API** (or **API Keys**). Copy the **Project URL** and the **anon / publishable** key. Both are safe for the browser.

- **11** In the Terminal, create the local settings file:

```
cp .env.example .env.local  
open -e .env.local
```

Replace the two placeholder values with your URL and anon key → save (⌘S) → close.

- **12** Tell Supabase where the school lives, so login emails link to it (not to localhost):

**Authentication** → **URL Configuration**.

- Site URL: `https://school.scribsandpixels.com` → **Save**
- Redirect URLs → **Add URL**, one at a time: `https://school.scribsandpixels.com/**`, `https://scribsandpixels-school.vercel.app/**`, `http://localhost:5173/**`

- **13** Give the live site the same two values: Vercel → the project → **Settings** → **Environment Variables**. Add `VITE_SUPABASE_URL` = your Project URL, and `VITE_SUPABASE_KEY` = the anon / publishable key (environments: all) → **Save**. Then **Deployments** → the top one → ⋮ → **Redeploy**.

- **14** Send Claude: Supabase project created and migrated. `.env.local` is filled in. (Don't paste any keys.)

**NEVER** copy the `service_role` key anywhere except the password manager. It bypasses all security. If it ever leaks: Supabase → API settings → roll (regenerate) it.

**EXPECTED OUTPUT:** The tables are visible in the Table Editor, and `.env.local` holds two real values.

**ON ERROR:** If `supabase db push` prints red text → copy the whole Terminal output (⌘A, ⌘C) → send it to Claude. Don't run it again repeatedly.

PROCEDURE `set_up_resend(domain)` → verified sending domain + API key

|               |                                                                                            |
|---------------|--------------------------------------------------------------------------------------------|
| <b>WHY</b>    | Resend sends the daily nudge and Sunday review emails, from an address on your own domain. |
| <b>INPUTS</b> | A registrar/DNS login (as in 06) and a password manager.                                   |
| <b>TIME</b>   | 20 minutes, then up to a few hours for verification                                        |

- **01** Go to `resend.com` → **Sign up** (the free plan is enough).
- **02** **Domains** → **Add Domain** → type your domain, e.g. `<yourdomain>` (or a subdomain like `mail.<yourdomain>`) → choose the region closest to you → **Add**.
- **03** Resend shows 3–4 DNS records (TXT, MX, DKIM). For **each** one: registrar → DNS → **Add record** → copy Type, Name and Value exactly → **Save**.
- **04** Back in Resend → **Verify DNS Records**. Repeat every 15 minutes until every row is green / **Verified**.
- **05** **API Keys** → **Create API Key**. Name: `school-nudges`. Permission: **Sending access** → **Add**.
- **06** Copy the key **now** (it starts with `re_` and is shown only once) → save it in your password manager as **SECRET** "Resend API key".
- **07** Decide the From address, e.g. `Scribs & Pixels <school@scribsandpixels.com>`. Write it here: \_\_\_\_\_
- **08** **OPTIONAL**, for login emails from your domain too: Supabase → **Authentication** → **Emails** → **SMTP Settings** → enable custom SMTP: host `smtp.resend.com`, port `465`, username `resend`, password = your Resend API key, sender = your From address → **Save**.

**EXPECTED OUTPUT:** The domain shows **Verified** in Resend, and the API key is in your password manager.

**ON ERROR:** If records won't verify after 24 hours → screenshot Resend's domain page and your DNS page, and send both. A common cause is the registrar adding your domain to the Name automatically (e.g. `send.yourdomain.com.yourdomain.com`): type only the part Resend shows before your domain.

## 09 • deploy\_email\_nudges()

ON HOLD

PROCEDURE deploy\_email\_nudges() → hourly emails running

|        |                                                                                                                                                   |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| WHY    | Switches on the daily nudge (Mon–Sat, at each learner's chosen hour, only if they haven't studied) and the Sunday review digest.                  |
| INPUTS | 07 and 08 done. Your project ref, Resend API key, From address, and the site address (APP_URL: <code>https://school.scribsandpixels.com</code> ). |
| TIME   | 20 minutes                                                                                                                                        |

- 01 Make a cron password in the Terminal and save the output in your password manager as **SECRET** "CRON\_SECRET":

```
openssl rand -hex 32
```

- 02 Set the four secrets. Paste this as **one** command after filling in the four placeholders; keep the quotes:

```
cd ~/Desktop/scribsandpixels-school
supabase secrets set RESEND_API_KEY="<re...>" EMAIL_FROM="Scribs & Pixels
<school@scribsandpixels.com>" APP_URL="https://school.scribsandpixels.com"
CRON_SECRET="<from step 1>"
```

- 03 Upload the email function:

```
supabase functions deploy send-nudges --no-verify-jwt
```

- 04 Do a safe dry run, which shows what *would* be sent without sending anything (replace both placeholders):

```
curl -s -X POST "https://<PROJECT_REF>.supabase.co/functions/v1/send-nudges?
dry-run" -H "Authorization: Bearer <CRON_SECRET>"
```

**IF** it prints JSON (even an empty list) → continue. **ELSE IF** it says `Unauthorized` → the `CRON_SECRET` doesn't match step 2; redo step 2.

- 05 In the project folder, open `supabase/setup/schedule-nudges.sql` in TextEdit. **Don't save changes to it**; you'll paste a filled-in copy instead.

- 06 Copy its whole content. In Supabase → **SQL Editor** → **New query** → paste.

- 07 In the pasted text only, replace `<project-ref>` with your project ref and `<CRON_SECRET>` with your cron secret.

- 08 Click **Run**.

- 09 Check it's scheduled: new query → `select jobname, schedule from cron.job;` → **Run**. You should see `send-nudges` with `0 * * * *`.

- 10 Send Claude: Nudges deployed and scheduled. Dry run returned `<paste the output>`.

**EXPECTED OUTPUT:** The cron job is listed and the dry run returns JSON. Real emails start once learners have accounts.

**ON ERROR:** If the SQL editor says `secret already exists`, you ran the script twice. Send Claude the message; don't try to delete vault secrets yourself.

## 10 · Appendix: words, tools and message templates

### Message template: error

```
ERROR in procedure <number + name>, step <number>.
What I did: <one sentence>
What I expected: <from EXPECTED OUTPUT>
What I saw: <exact text or "see screenshot">
Device/browser: <e.g. iPhone 13 Safari>
```

### Message template: bugs from a test session

```
TEST SESSION <date>, course <Fullstack | French | Excel>, device <...>
1. [BROKEN|WRONG|CONFUSING|UGLY|SLOW] page #/<address after #/>
   What happened: ...
   What I expected: ...
2. ...
```

### Words

|                      |                                                                                                                                                                           |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Repo</b>          | The project's folder on GitHub:<br><code>github.com/VictorAnimasahun/scribsandpixels-school</code> . Claude pushes every finished change to its <code>main</code> branch. |
| <b>Deploy</b>        | Putting the latest version on the internet. GitHub Pages does it automatically about 2 minutes after each push.                                                           |
| <b>DNS / record</b>  | The internet's address book. A CNAME record says "this name points to that server".                                                                                       |
| <b>Registrar</b>     | The company you bought the domain from. That's where DNS records are edited.                                                                                              |
| <b>Migration</b>     | A file that creates database tables. Running it twice is safe for Supabase; it skips what's done.                                                                         |
| <b>Secret / key</b>  | A password for a service. "anon/publishable" keys are safe in the browser. "service_role", the Resend API key, CRON_SECRET and the DB password are <b>SECRET</b> .        |
| <b>Edge function</b> | A small program that runs on Supabase's servers. Here: <code>send-nudges</code> , the email sender.                                                                       |
| <b>Cron</b>          | A timer. <code>0 * * * *</code> means "every hour, on the hour".                                                                                                          |
| <b>Dry run</b>       | A rehearsal that shows what would happen without doing it.                                                                                                                |

### Where to find things

|                              |                                                                                                                                                                                 |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>The school</b>            | <code>school.scribsandpixels.com</code> (main; Vercel). Copies: <code>scribsandpixels-school.vercel.app</code> , <code>victoranimasahun.github.io/scribsandpixels-school</code> |
| <b>To-do list (Claude's)</b> | <code>TOD0.md</code> in the repo                                                                                                                                                |

|                               |                                                                                           |
|-------------------------------|-------------------------------------------------------------------------------------------|
| <b>Project rules</b>          | <code>CLAUDE.md</code> in the repo                                                        |
| <b>Phase 2 draft</b>          | <code>src/content/courses/fullstack-ml/phase-2-plan.md</code>                             |
| <b>This document's source</b> | <code>docs/your-tasks/your-tasks.html</code> . Ask Claude to update it when tasks change. |